Universidad Nacional Autónoma de México

Facultad de Ingeniería

Semestre: 2025-2

Asignatura: Lenguajes Formales y Autómatas

Grupo: 1

Profesor: Antonio Montalvo García

Alumno: Dueñas Fuentes David

No. Cuenta: 321029988

Fecha de Entrega: 05 de junio de 2025

Introducción

En la asignatura de Lenguajes Formales y Autómatas abordamos temas como un Autómatas Finitos Determinísticos, Autómatas Finitos No Determinísticos, Autómatas Push-Down, etc. Cada uno de los tipos de autómatas vistos tienen su propia gramática y por ende tienen sus propias reglas de producción y un lenguaje específico. Como todo lo anterior fue visto de manera teórica ahora se necesita implementar de manera práctica.

Para implementar la teoría de manera práctica se creará un analizador léxico-sintáctico en Java el cual necesitara de dos herramientas del lenguaje JFlex y CUP, las cuales se describen brevemente a continuación:

- **JFlex:** Es un **generador de analizadores léxicos** (también conocido como generador de escáneres) para Java. Su función es tomar una especificación de patrones (expresiones regulares) y las acciones asociadas a cada patrón, y generar un programa (un lexer) que lee un flujo de entrada (por ejemplo, código fuente) y lo divide en unidades lógicas llamadas "tokens". Los tokens son los componentes básicos del lenguaje, como palabras clave, identificadores, operadores, números, etc.
- **CUP (Construction of Useful Parsers):** Es un **generador de analizadores sintácticos LALR** para Java. Una vez que JFlex ha generado los tokens, CUP entra en juego. Toma una gramática y genera un programa (un parser) que verifica si la secuencia de tokens generada por el lexer se ajusta a la gramática del lenguaje. En otras palabras, se asegura de que el código fuente esté bien formado sintácticamente. Si el código cumple con las reglas, el parser puede construir una representación interna del mismo (como un árbol de sintaxis abstracto) para su posterior procesamiento.

Objetivo

La realización de este proyecto pretende implementar un analizador léxico-sintáctico en Java a partir de un lenguaje reducido de programación en C que en específico analice los tres tipos de estructura de repetición que existen en el lenguaje for, while y do-while. Para esto se utilizará una gramática libre de contexto y el apoyo de las herramientas de Java, JFlex y CUP.

Limitaciones y alcances

El analizador léxico contiene los tokens siguientes:

TOKENS
Linea
Comillas
T_dato
Cadena
If
Else
Do
While
For
Include
Printf
Scanf
Main
Igual

Suma
Resta
Multiplicacion
Division
Modulo
Op_incremento
Op_logico
Op_relacional
Op_atribucion
Op_booleano
Parentesis_a
Parentesis_c
Llave_a
Llave_c
Corchete_a
Corchete_c
P_coma
Numeral
Dolar
Backslash
Punto
Dos_puntos
Interrogacion_c
Interrogacion_a
Exclamacion_a
Identificador
Numero
ERROR

Lo que nos quiere decir que el analizador léxico solo identificará los caracteres que se encuentren dentro de estos 42 tokens, más sin embargo como explicaré a continuación esto no quiere decir que el analizador sintáctico contendrá en su gramática todos y cada uno de estos tokens.

Para el analizador sintáctico encontramos:

Alcances	Limitaciones

Lenguaje de entrada:
 	
• Soporta archivos fuente en C **simplificados**:
  ○ Inclusión de librerías con #include.
  ○ Declaración de la función principal int main() { … }.
  ○ Estructuras de control básicas:
  ▪ if, if-else
  ▪ while, do-while
  ▪ for
  ○ Declaración y asignación de variables simples de tipo básico (int, float, etc., representados como T_dato).
  ○ Expresiones relacionales simples.
  ○ Asignaciones simples.
	Lenguaje no soportado completamente:
	
	• No soporta:
	  ○ Tipos de datos compuestos (struct, union).
	  ○ Punteros y operaciones de punteros.
	  ○ Declaraciones múltiples de variables en una línea (int a, b, c;).
	  ○ Expresiones booleanas compuestas con operadores lógicos (&&, ||).
	  ○ Definición de funciones con múltiples parámetros.

• Permite declarar funciones con **un único parámetro** como máximo, aunque el cuerpo está vacío ({ }).	○ Comentarios (// o /* ... */). ○ Directivas del preprocesador más complejas (#define, #ifdef, etc.). ○ Strings y operaciones con strings (más allá de cadenas constantes).
Ámbito de validación: • Sintaxis correcta de las sentencias. • Uso correcto de paréntesis, llaves y operadores básicos. • Verificación de la estructura básica de control.	**Errores semánticos no detectados:** • No detecta: ○ Tipos incompatibles (ej.: int x = "hola";). ○ Declaración de variables duplicadas. ○ Uso de variables no declaradas. ○ Asignaciones fuera de ámbito. ○ Tipos de retorno de funciones.
Salidas esperadas: • Informe de errores sintácticos con línea y columna. • Tokens correctamente reconocidos por el analizador léxico (JFlex). • Estructura sintáctica reconocida por CUP.	**Analizador léxico y sintáctico:** • No maneja el main() con parámetros (int main(int argc, char *argv[])). • La posición de error puede ser imprecisa en yycolumn. • No maneja expresiones más complejas (e.g. ternarios, encadenamientos de operaciones lógicas).

Como se mencionó en el objetivo nos centraremos en el análisis de estructuras de repetición for, while y do-while.

Desarrollo

Antes de iniciar con la descripción de los 6 archivos que en esencia componen el proyecto es necesario mencionar que se deben de incluir las librerías de JFlex y CUP en el proyecto de NetBeans, dichas librerías las podemos descargar de internet, pero de todas formas estarán citadas en la bibliografía. De igual manera es preciso mencionar que se puede decir que el proyecto contiene dos analizadores léxicos, el primero que solo analiza tokens y el segundo es el que forma parte del analizador sintáctico.

Entrando de lleno con el proyecto a este lo componen los archivos Lexer.flex, LexerCup.flex, Principal.java, Sintax.cup, Tokens.java y FrmPrincipal.java. A continuación, se menciona de lo que es encargado cada archivo.

Principal.java

Este es el corazón del proyecto pues es desde su método generar que a partir de los archivos .flex y .cup es que se generan los archivos .java procedentes de JFlex y CUP los cuales contiene las reglas gramaticales y las regla de lectura de tokens.

Definición de Rutas:

- ruta1: Apunta a un archivo .flex llamado Lexer.flex. Este archivo contiene las reglas para definir los tokens del lenguaje.
- ruta2: Apunta a otro archivo .flex llamado LexerCup.flex. Este archivo contiene las mismas reglas que el anterior solo que con las declaraciones necesarias para integrarse con CUP.
- rutaS: Es un arreglo de cadenas que contiene argumentos para CUP. -parser Sintax indica que el nombre de la clase del parser generado será Sintax, y la última cadena es la ruta al archivo .cup (Sintax.cup), que contiene la gramática del lenguaje.

Generación con JFlex:

- JFlex.Main.generate(archivo); se invoca dos veces, una para ruta1 y otra para ruta2. Esto hace que JFlex lea las definiciones de los archivo .flex y **genere los archivos Java correspondientes** (probablemente Lexer.java y LexerCup.java), que serán las clases del analizador léxico.

Generación con CUP:

- java_cup.Main.main(rutaS); ejecuta la herramienta CUP. CUP lee el archivo Sintax.cup (que define la gramática del lenguaje) y **genera dos archivos Java**:
    - Sintax.java: Esta es la clase del **analizador sintáctico** (el parser).
    - sym.java: Este archivo contiene las **constantes de los símbolos terminales y no terminales** definidos en la gramática, que son utilizados tanto por el lexer como por el parser para referenciar los diferentes tipos de tokens y reglas gramaticales.

Organización de Archivos Generados:

- Después de que JFlex y CUP generan los archivos, es común que los coloquen en el directorio raíz del proyecto o en un lugar predeterminado. Este bloque de código se encarga de **mover esos archivos generados a la ubicación correcta** dentro del proyecto (el paquete codigo en src):
    - Verifica si sym.java ya existe en el destino y lo elimina si es así.
    - Mueve6 sym.java de la raíz del proyecto a src/codigo/sym.java.

Tokens.java

Define manualmente en una clase enum los 42 tokens que contemplará el analizador léxico.

Lexer.flex

Este archivo. flex es una especificación para el analizador léxico (lexer) escrita para la herramienta JFlex.

El archivo se divide en dos secciones, separadas por %%:

1. Declaraciones de usuario y configuración (Primera sección antes del primer %%):
    - package codigo;: Declara el paquete Java al que pertenecerá la clase generada.
    - import static codigo.Tokens.*;: Importa estáticamente todos los miembros de la clase Tokens. Esto es crucial, ya que Tokens contiene las constantes de los tipos de tokens que el lexer debe devolver.
    - %class Lexer: Define el nombre de la clase Java que JFlex generará para el analizador léxico. En este caso, la clase se llamará Lexer.
    - %type Tokens: Especifica el tipo de valor que el método yylex() (el método principal del lexer que devuelve el siguiente token) retornará. Aquí, indica que retornará un objeto de tipo Tokens.
    - Macros (Expresiones regulares):
        - L=[a-zA-Z_]+: Define una macro L que representa una o más letras (mayúsculas o minúsculas) o guiones bajos. Esto es útil para definir identificadores.
        - D=[0-9]+: Define una macro D para una o más dígitos. Esto es para números.
        - espacio=[,\t,\r]+: Define una macro espacio para uno o más caracteres de espacio (espacio en blanco, tabulador, retorno de carro).
    - Bloque de código Java (%{ ... %}):
        - public String lexeme;: Declara una variable de instancia lexeme dentro de la clase Lexer generada. Esta variable se usará para almacenar el texto real que coincide con un token (por ejemplo, si el token es un identificador, lexeme guardará el nombre de ese identificador).
2. Reglas léxicas (Segunda sección entre los dos %%): Esta es la parte más importante, donde se definen las reglas para reconocer los tokens. Cada regla tiene dos componentes:
    - Una expresión regular: Describe el patrón de texto que el lexer debe buscar.
    - Una acción Java: Es el código que se ejecuta cuando el lexer encuentra un texto que coincide con la expresión regular.

LexerCup.flex

Este archivo .flex, es una especificación para JFlex que tiene un objetivo muy específico: generar un analizador léxico (lexer) que sea compatible y trabaje directamente con CUP (Construction of Useful Parsers).

Al igual que el archivo anterior se compone de dos partes:

1. Sección de Configuraciones y Declaraciones (antes del primer %%)

- package codigo;: Especifica que la clase Java generada (LexerCup) pertenecerá al paquete codigo.
- import java_cup.runtime.Symbol;: ** crucial para la integración con CUP**. Importa la clase Symbol de la librería java_cup.runtime. Los tokens que este lexer devuelva serán instancias de esta clase.
- %class LexerCup: Define el nombre de la clase Java que JFlex generará. En este caso, será LexerCup.
- %type java_cup.runtime.Symbol: Indica que el método yylex() del lexer (el que devuelve el siguiente token) retornará un objeto de tipo java_cup.runtime.Symbol. Esto es fundamental para que CUP pueda entender la salida del lexer.
- %cup: Esta directiva es explícita para JFlex e indica que el lexer está siendo generado para ser usado con CUP. JFlex ajusta el código generado para cumplir con las expectativas de CUP.
- %full: Es una directiva de JFlex que asegura que el analizador léxico generado es "completo", cubriendo todos los posibles estados del autómata.
- %line: Habilita el seguimiento del **número de línea** actual en el código fuente. Se accede a él a través de la variable yyline.
- %char: Habilita el seguimiento del **número de columna** actual en la línea. Se accede a él a través de la variable yycolumn.
- L=[a-zA-Z_]+, D=[0-9]+, espacio=[,\t,\r,\n]+: Estas son **macros de expresiones regulares**, iguales a las del otro archivo. Definen patrones reutilizables para letras, dígitos y espacios en blanco (que ahora incluyen el salto de línea \n).
- Bloque de código Java (%{ ... %}):
    - private Symbol symbol(int type, Object value) y private Symbol symbol(int type): Estas son dos funciones auxiliares (métodos privados) que se incluyen en la clase LexerCup generada. Su propósito es **crear instancias de java_cup.runtime.Symbol**.
        - La versión con value se usa para tokens que tienen un valor asociado (como el texto de un identificador o el valor de un número).
        - La versión sin value se usa para tokens que no necesitan un valor específico, solo su tipo (como un paréntesis o un operador).
        - Ambas calculan el inicio y el final de la posición del token en la línea (yycolumn y yytext().length()).
        - **Importante:** Aunque estas funciones están definidas, veremos que en la sección de reglas léxicas **no se están usando directamente**. En su lugar, los Symbol se crean explícitamente en cada regla (new Symbol(sym.TipoToken, yyline, yycolumn,

yytext())). Las funciones symbol podrían ser útiles si se quisiera encapsular la lógica de creación del Symbol o si el diseño inicial contemplaba un uso diferente.

2. Sección de Reglas Léxicas (entre los dos %%)

Esta sección define cómo el lexer identifica y devuelve cada token, pero ahora lo hace encapsulando la información del token en un objeto Symbol de CUP.

- {espacio} {/*Ignore*/} y ("//"(.)*) {/*Ignore*/}: Ignoran los espacios en blanco y los comentarios de una sola línea, tal como en el otro lexer.
- Reglas para tokens con valores: Para la mayoría de los tokens que representan palabras clave, operadores, identificadores, números, etc., la acción Java asociada es:
    - return new Symbol(sym.TipoToken, yyline, yycolumn, yytext());
    - sym.TipoToken: Aquí sym es la clase generada por CUP (normalmente sym.java) que contiene las constantes estáticas para cada tipo de token (por ejemplo, sym.Comillas, sym.T_dato, sym.Identificador). Es la forma en que el lexer y el parser se "hablan" sobre qué tipo de token se ha encontrado.
    - yyline: La línea donde se encontró el token.
    - yycolumn: La columna donde comienza el token en la línea.
    - yytext(): El texto real que coincidió con la expresión regular (el "lexeme" del token).

Sintax.cup

Este archivo .cup (similar a los archivos YACC/Bison), es una especificación de gramática para la herramienta CUP (Construction of Useful Parsers). Su propósito es definir la estructura sintáctica de un lenguaje de programación (similar a un subconjunto simplificado de C o Java) y, a partir de ella, generar un analizador sintáctico (parser).

Podemos definir 5 partes importantes de este archivo:

1. Código Java para el Parser
   Esta sección parser code permite inyectar código Java directamente en la clase parser que CUP generará. Aquí, se define:
   - Una variable s de tipo Symbol para almacenar el símbolo donde ocurrió un error de sintaxis.
   - El método syntax_error(Symbol s): Este método se sobrescribe para que CUP lo llame automáticamente cada vez que encuentre un **error de sintaxis** durante el análisis. Almacena el Symbol (el token) donde ocurrió el error.
   - El método getS(): Un método para obtener el Symbol del último error de sintaxis detectado. Esto es útil para reportar errores detallados al usuario.

2. Declaración de Terminales (Tokens)

Aquí se listan todos los **símbolos terminales** del lenguaje. Estos son los tokens que el analizador léxico (generado por JFlex) reconocerá y pasará al parser. Los nombres aquí deben coincidir con las constantes definidas en el archivo sym.java que CUP genera (por ejemplo, sym.Comillas, sym.T_dato).

3. Declaración de No Terminales
   Estos son los **símbolos no terminales** de la gramática. Representan construcciones abstractas del lenguaje que se componen de otros terminales y/o no terminales (por ejemplo, un PROGRAMA está compuesto por LISTA_INCLUDES, FUNCION y MAIN_FUNC). Son los "bloques de construcción" lógicos de la sintaxis.

4. Símbolo de Inicio de la Gramática
   PROGRAMA es el **símbolo inicial** de la gramática. Esto significa que el parser intentará reconocer una secuencia de tokens que, en última instancia, pueda reducirse a un PROGRAMA completo.

5. Reglas de Producción de la Gramática
   El resto del archivo define las **reglas de producción (o reglas de la gramática)**. Cada regla especifica cómo un no terminal puede ser descompuesto en una secuencia de terminales y/o otros no terminales. Son la columna vertebral del lenguaje.

FrmPrincipal.java.

Este código de Java crea una **ventana de aplicación (una interfaz gráfica de usuario o GUI)** para demostrar el funcionamiento del **analizador léxico** y del **analizador sintáctico**. Permite a un usuario escribir o cargar código y ver cómo se descompone en tokens (análisis léxico) y si su estructura es correcta (análisis sintáctico).

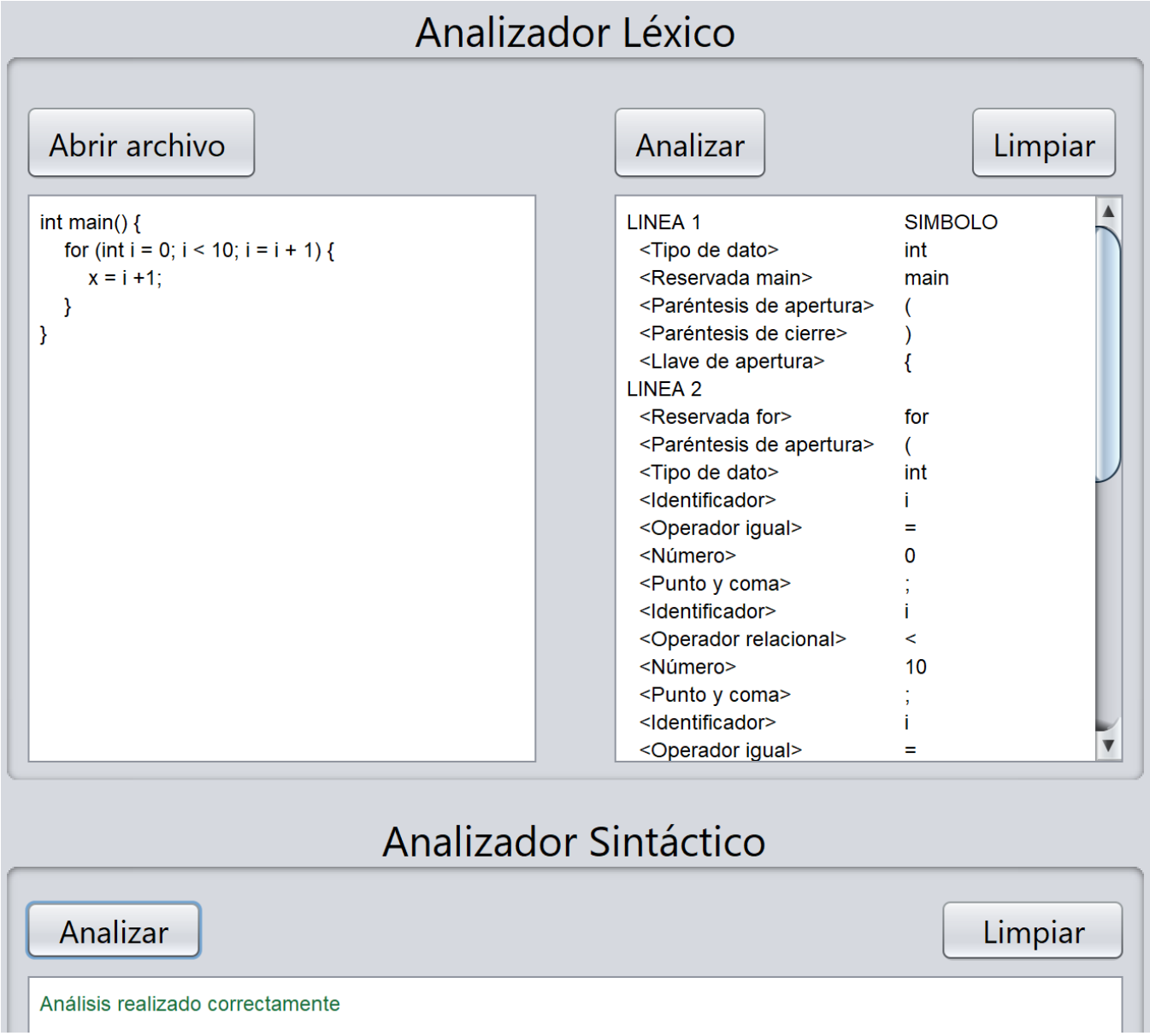
En pocas palabras, hace lo siguiente:

1. **Crea una ventana:** Es la interfaz principal de la aplicación.
2. **Tiene un área de texto grande (txtLectura):** Aquí es donde el usuario puede escribir el código que quiere analizar o cargar un archivo con código.
3. **Tiene dos paneles para resultados:**
   - Uno para el **Análisis Léxico** (txtResultado): Mostrará una lista de los "tokens" (las palabras y símbolos individuales) que el programa encuentra en el código.
   - Otro para el **Análisis Sintáctico** (txtResultadoS): Indicará si la estructura del código es correcta o, en caso de error, dónde se encontró un problema de sintaxis.
4. **Botones para interactuar:**
   - **"Abrir archivo" (btnAbrir):** Permite al usuario cargar un archivo de texto (que contiene código) en el área de txtLectura.
   - **"Analizar" (btnAnalizar):** Cuando se presiona este botón, el programa toma el texto de txtLectura y realiza el **análisis léxico**. Muestra los tokens identificados en txtResultado.

- **"Limpiar" (btnLimpiar):** Borra el contenido del área de resultados del análisis léxico.
- **"Analizar" (btnAnalizarS en el panel sintáctico):** Al presionarlo, el programa toma el texto de txtLectura y realiza el **análisis sintáctico**. Muestra el resultado (éxito o error) en txtResultadoS.
- **"Limpiar" (btnLimpiarS):** Borra el contenido del área de resultados del análisis sintáctico.

Resultados

Análisis léxico y sintáctico de un ciclo for básico en C



Análisis léxico y sintáctico de un ciclo while básico en C

Analizador Léxico

Abrir archivo | Analizar | Limpiar

```
#include "stdio.h"

int main() {
    while (x < 10) {
        x = x + 1;
    }
}
```

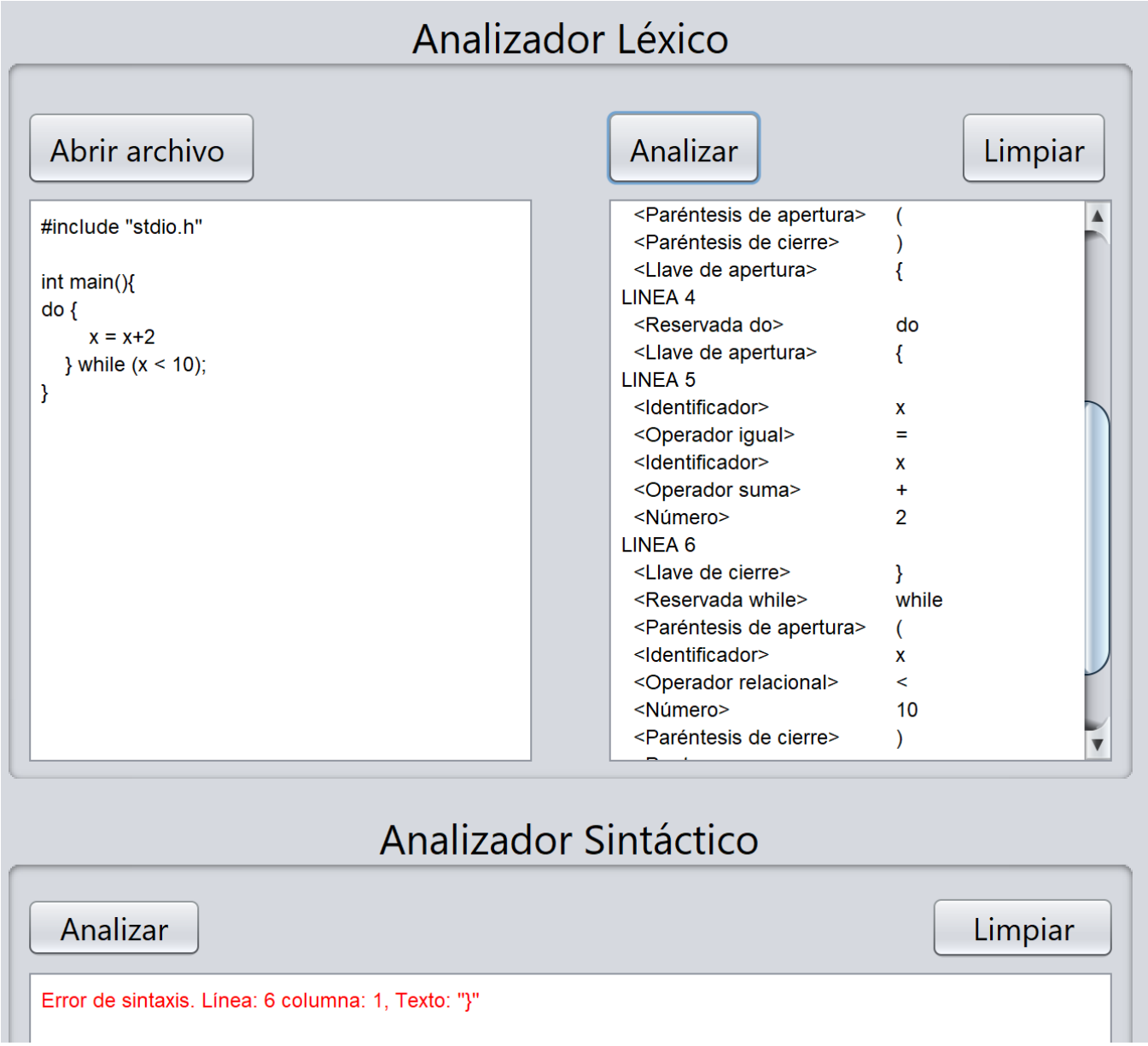
LINEA 1	SIMBOLO
<Hashtag o numeral>	#
<Palabra reservada include>	include
<Comillas>	"
<Identificador>	stdio
<Punto>	.
<Identificador>	h
<Comillas>	"
LINEA 2	
LINEA 3	
<Tipo de dato>	int
<Reservada main>	main
<Paréntesis de apertura>	(
<Paréntesis de cierre>	)
<Llave de apertura>	{
LINEA 4	
<Reservada while>	while
<Paréntesis de apertura>	(
<Identificador>	x
<Operador relacional>	<

Analizador Sintáctico

Analizar | Limpiar

Análisis realizado correctamente

Análisis léxico y sintáctico de un ciclo do - while básico en C



Conclusión

El desarrollo de este proyecto sin duda significó un reto para mí. En un principio creí que sería algo fácil de hacer, pero en realidad no fue del todo así, debido a que a lo largo de su construcción me encontré con varios problemas desde la no detección de algunos caracteres, la creación de un pequeño lenguaje libre de contexto que no tuviera ambigüedades hasta errores en las coordenadas al momento de encontrar un error sintáctico. La idea principal del proyecto era crear un analizador sintáctico que funcionara para un subconjunto del lenguaje de programación C, o sea un mini C pero al enfrentarme a los problemas antes

mencionados se decidió delimitar el proyecto solo a las tres estructuras de repetición del lenguaje. Con esta delimitación se logró llegar a un resultado considerablemente bueno y se implementaron en Java varios conceptos teóricos vistos a lo largo del curso.

Bibliografía

OpenAI. (2024, marzo 15). *Introducing GPT-4 Turbo* [Video]. YouTube. https://www.youtube.com/watch?v=4Z6Tnit810Y

Autor desconocido. (s.f.). *Archivo almacenado en Mega* [Archivo]. Recurso no recuperable públicamente. https://mega.nz/file/kp5wmSTR#VRGITdbm5zuSWDMeZ8hVgVYWCNl-J0gMcAqQ2k8FBfM

Autor desconocido. (s.f.). *Archivo almacenado en Mega* [Archivo]. Recurso no recuperable públicamente. https://mega.nz/file/15Zk2IhJ#0_IRdD6uQWF02_4MWNkbz-JwR8d4222Pz3OQh5SIhec

Autor desconocido. (s.f.). *Archivo almacenado en Mega* [Archivo]. Recurso no recuperable públicamente. https://mega.nz/file/h4By1ahD#HwNXmtmw3BlUlAPNNilX1Am6SDICMvA8LOTkDPYZg4I